

National Textile University, Faisalabad



Department of Computer Science

Name:	Areeba Mohsin
Class:	BSCS – 5B
Registration No:	23-NTU-CS-1138
Course Name:	Embedded IOT system
Submitted To:	Nasir Mehmood
Submission Date:	21st Dec, 2025

Homework-01

Embedded IOT Systems Fall2025

CS(A-B) AI 5th

Question-1 ESP32 Webserver (webserver.cpp)

Part A: Short Questions

1. What is the purpose of `WebServer server(80);` and what does port 80 represent?

The purpose of `WebServer server(80);`

Through this line, you are telling the ESP32 that: "I want to run a website."

80 means that your ESP32 is ready to display the website in the standard way. Whenever someone enters the IP address of your ESP32 on a phone or laptop, the ESP32 will know that 'someone has come to Port 80, I need to show them the Temperature page

2. Explain the role of `server.on("/", handleRoot);` in this program.

`"/"` means the main path (Root) of the website, which is the first page that appears when opened.

`handleRoot` is the name of the function that prepares the HTML page for temperature and humidity and sends it to the browser.

It works like a "Traffic Signal" that activates the correct function as soon as a request comes in.

3. Why is `server.handleClient();` placed inside the `loop()` function? What will happen if it is removed?

This is placed inside the `loop()` so that the ESP32 continuously checks whether someone has opened the website in the browser or not.

If you remove it from the loop, then when you enter the IP address in the browser, the website will not load. The browser will show "Connection Timed Out". It only focus on dht sensor

4. In `handleRoot()`, explain the statement: `server.send(200, "text/html", html);`

This line tells the browser with an "OK" signal (200) that the data being sent is a web page (text/html). Its main job is to send all the temperature data and design stored in the `html` variable to the browser to display on the screen.

5. What is the difference between displaying last measured sensor values and taking a fresh DHT reading inside handleRoot()?

Displaying Last Values: when you press button then the database save the values; when we open web page then saved readings are displays. The limitation is that if you open the page after an hour, we will see the old readings.

Fresh Reading: In this when we open webpage it calls to take fresh reading so we get fresh readings; it may be slower as it takes some seconds

. Part B: Long Question

Describe the complete working of the ESP32 webserver-based temperature and humidity monitoring system.

1. Starting:

When the ESP32 is turned on, it first starts the OLED display and the DHT sensor. After that, it connects to your WiFi (**Wokwi-GUEST**) and generates its own IP address (like 192.168.x.x). This IP address is shown on the OLED screen and the Serial Monitor.

2. Button Press:

This system is "Event-based." Meaning, the sensor does not constantly read data; instead, when you press the physical button. The ESP32 runs the readDHTValues () function. It takes the latest temperature and humidity from the DHT sensor. This data is saved in the variables last Temp and last Humidity and is immediately updated on the OLED screen.

3. Background:

In the background, server.handleClient() continuously checks for browser requests. When you enter the ESP32's IP address in the browser on your phone or laptop. The browser requests the main page from the ESP32. The ESP32 runs handleRoot() function.

4. Sending:

The handleRoot() function creates an HTML page that contains the saved temperature and humidity data. This page also has an automatic refresh command (content='5') that tells the browser to refresh the page every 5 seconds. Finally, the entire page is sent to the browser via server.send(), and the data starts appearing on the screen.

Question-2 Blynk Cloud Interfacing (blynk.cpp)

Part-A: Short Questions

1. What is the role of Blynk Template ID in an ESP32 IoT project? Why must it match the cloud template?

The Blynk Template ID is like a Blueprint for your project. It tells the cloud what your device is (ESP32), how many sensors it has, and how the widgets (gauges/charts) will appear on the dashboard.

If the cloud's ID and the code's ID do not match, then Blynk will not show any data

2. Differentiate between Blynk Template ID and Blynk Auth Token.

Feature	Blynk Template ID	Blynk Auth Token
Purpose	It indicates the device's Model or design.	It is the device's Unique Key that allows it to log in.
Scope	One ID can be used by many devices of the same type.	Each device has its own separate and unique token.
Function	Sets up the dashboard and interface.	Connects the device securely to the server.

3. Why does using DHT22 code with a DHT11 sensor produce incorrect readings? Mention one key difference between the two sensors.

The reading will be incorrect as the code for both of the DHTs will be different because of their different data formats. Moreover DHT 22 is more accurate than that of DHT 11.

4. What are Virtual Pins in Blynk? Why are they preferred over physical GPIO pins for cloud communication?

Virtual pins like V0, V1 are like communication bridge between esp and BLYNK.GPIO pins are limited but virtual pin have more range i.e.: 0 to 255.

Benefit: there is no need to reset or change wiring of physical circuit.

5. What is the purpose of using BlynkTimer instead of delay() in ESP32 IoT applications?

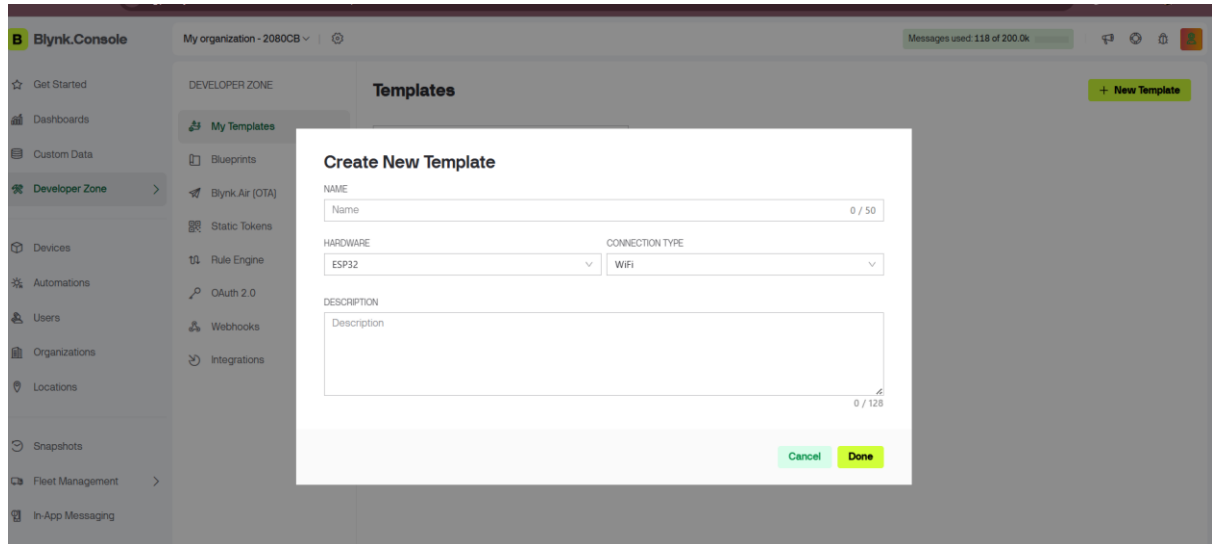
Using delay() in IoT projects less suitable as delay stops the processor and the whole program will be held until the delay timer finishes.

BlynkTimer is more suitable as it doesnot held the processor. It runs task for fixed time without stopping ESP .even the BLYNK web stays online.

Part-B: Long Question

Explain the complete workflow of interfacing ESP32 with Blynk Cloud to display temperature and humidity values.

- First, we create a new template in the Blynk app.



- From the template, we copy the Auth Token and paste it into the ESP32 code.

```

8
9
10 #define BLYNK_TEMPLATE_ID "TMPL6DqnFEf_K"
11 #define BLYNK_TEMPLATE_NAME "DHT bugoff"
12 #define BLYNK_AUTH_TOKEN "KR5qkGXONtM_sLbiMfMzmEBONQ8oJz__"
13 #define BLYNK_PRINT Serial
14

```

- Then, in the code, we write the WiFi SSID and password of the network we want to connect to.
- After that, we write the circuit and sensor code and set a timer for reading the sensor.
- In the setup() function, we start Blynk and initialize the sensor and timer. In the loop() function, we call Blynk and the timer function.
- The sensor values are read inside a function, and by using virtualWrite, the values are sent to Blynk Cloud.
- The readings update automatically, either by pressing a button or after every 5 seconds, because a periodic function is used and called in the setup.

